

UNIVERSIDAD TECNOLÓGICA NACIONAL
Facultad Regional San Nicolás

**Tecnicatura Universitaria en Programación a
Distancia**

Programación I

TRABAJO PRÁCTICO INTEGRADOR

**"Gestión de Datos de Países en Python:
Filtros, Ordenamientos y Estadísticas"**

Alumno:

Aaron Axel Cambursano

INDICE

Introducción	3
Marco Teórico	3
Decisiones técnicas y arquitectura	7
Dificultades técnicas y conclusiones	12
Bibliografía	15

TRABAJO PRÁCTICO INTEGRADOR

Gestión de Datos de Países mediante Python y Archivos CSV

1. Introducción

El presente Trabajo Práctico Integrador tiene como finalidad aplicar de manera práctica los conocimientos adquiridos durante la cursada de Programación I mediante el desarrollo de un sistema de gestión de información utilizando el lenguaje Python.

El programa desarrollado permite administrar una base de datos de países almacenada en un archivo CSV. A través de un menú interactivo, el usuario puede realizar diferentes operaciones, como agregar nuevos registros, actualizar información existente, realizar búsquedas, aplicar filtros, ordenar los datos según distintos criterios y obtener estadísticas generales sobre la información almacenada.

Para el desarrollo del sistema se implementaron conceptos fundamentales de programación, entre ellos el uso de funciones, estructuras condicionales, estructuras repetitivas, listas, diccionarios, manejo de excepciones y lectura y escritura de archivos CSV.

Uno de los principales objetivos durante la implementación fue mantener un código modular y organizado. Para ello, cada funcionalidad del sistema fue desarrollada mediante una función independiente, facilitando tanto la comprensión del programa como su mantenimiento y futuras ampliaciones.

Este proyecto permitió integrar los contenidos desarrollados durante la asignatura en una aplicación funcional, fortaleciendo las habilidades de resolución de problemas, análisis lógico y programación estructurada.

2. Marco Teórico

2.1 Lenguaje Python

Python es un lenguaje de programación de alto nivel, interpretado y multiparadigma, ampliamente utilizado debido a su sintaxis sencilla y legible. Permite desarrollar aplicaciones de escritorio, sistemas web, automatización de tareas, análisis de datos e inteligencia artificial, entre muchas otras aplicaciones.

Una de sus principales ventajas es la gran cantidad de bibliotecas disponibles y la facilidad para trabajar con distintos tipos de estructuras de datos, lo que lo convierte en una excelente herramienta para el aprendizaje de la programación.

En este proyecto Python fue utilizado para desarrollar toda la lógica del sistema, implementar el menú de opciones, validar los datos ingresados por el usuario y gestionar el almacenamiento permanente de la información.

2.2 Funciones

Las funciones constituyen bloques de código reutilizables que permiten organizar un programa dividiéndolo en tareas específicas. Cada función realiza una operación determinada y puede ser invocada desde cualquier parte del programa cuando resulta necesaria.

La modularización mediante funciones aporta importantes ventajas:

- Reduce la repetición de código.
- Facilita el mantenimiento del programa.
- Mejora la legibilidad.
- Permite reutilizar procedimientos.

En este trabajo se desarrolló una función independiente para cada una de las funcionalidades solicitadas por la consigna, entre ellas:

- `agregar_pais()`
- `actualizar_pais()`
- `buscar_pais()`
- `filtrar_por_continente()`
- `filtrar_por_poblacion()`
- `filtrar_por_superficie()`
- `ordenar_por_nombre()`
- `ordenar_por_poblacion()`
- `ordenar_por_superficie()`
- `mostrar_estadisticas()`

Además, se implementaron funciones auxiliares para validar los datos ingresados por el usuario antes de incorporarlos al sistema.

2.3 Listas

Las listas son estructuras de datos ordenadas que permiten almacenar múltiples elementos dentro de una misma variable. Una de sus principales características es que pueden modificarse dinámicamente, agregando o eliminando elementos durante la ejecución del programa.

Dentro del sistema desarrollado, las listas fueron utilizadas para almacenar temporalmente los registros leídos desde el archivo CSV. Esto permitió recorrer

posteriormente todos los países para realizar búsquedas, filtrados, ordenamientos y cálculos estadísticos.

Durante la actualización de un país también se utilizó una lista para reconstruir completamente el archivo CSV luego de modificar el registro correspondiente.

2.4 Diccionarios

Los diccionarios son estructuras de datos compuestas por pares clave-valor. Cada dato se almacena asociado a una clave que permite acceder rápidamente a la información.

En este proyecto, la biblioteca csv permitió leer cada fila del archivo mediante DictReader(), generando un diccionario para cada país con las siguientes claves:

- nombre
- poblacion
- superficie
- continente

Gracias al uso de diccionarios fue posible acceder directamente a cada dato utilizando expresiones como:

```
fila["nombre"]
```

```
fila["poblacion"]
```

Asimismo, durante el cálculo de estadísticas se implementó un diccionario adicional para contabilizar la cantidad de países pertenecientes a cada continente.

2.5 Archivos CSV

Un archivo CSV (Comma Separated Values) es un formato de almacenamiento de información donde cada fila representa un registro y cada columna un atributo.

En este proyecto el archivo paises.csv fue utilizado como base de datos del sistema, permitiendo almacenar permanentemente la información de cada país.

Para manipular este archivo se empleó el módulo csv de Python, utilizando:

- csv.DictReader(), para leer los registros.
- csv.DictWriter(), para actualizar el archivo.
- csv.writer(), para agregar nuevos registros.

El uso de archivos CSV permitió conservar la información aun cuando el programa finaliza su ejecución.

2.6 Estructuras Condicionales

Las estructuras condicionales permiten ejecutar diferentes bloques de instrucciones dependiendo del cumplimiento de una condición lógica.

En el sistema fueron utilizadas principalmente mediante la sentencia if-elif-else para:

- Validar datos ingresados por el usuario.
- Detectar países duplicados.
- Comprobar la existencia de un país.
- Filtrar registros.
- Controlar las opciones del menú principal.

Su utilización permitió controlar el flujo de ejecución del programa y evitar operaciones inválidas.

2.7 Estructuras Repetitivas

Las estructuras repetitivas permiten ejecutar un conjunto de instrucciones múltiples veces.

En este proyecto se utilizaron principalmente:

- while, para mantener activo el menú principal y repetir el ingreso de datos hasta obtener valores válidos.
- for, para recorrer todas las filas del archivo CSV, ordenar registros, calcular estadísticas y realizar búsquedas.

Estas estructuras fueron fundamentales para el procesamiento de la información almacenada.

2.8 Manejo de Excepciones

El manejo de excepciones permite controlar errores durante la ejecución del programa evitando su finalización inesperada.

Para ello se utilizaron bloques try-except que verifican el ingreso correcto de datos por parte del usuario.

Entre las validaciones implementadas se encuentran:

- Ingreso exclusivo de texto para nombres y continentes.
- Ingreso de números enteros positivos para población y superficie.
- Control de rangos válidos para los filtros.
- Prevención de errores de conversión mediante ValueError.

Gracias a estas validaciones el programa resulta más robusto y confiable.

3. Decisiones Técnicas y Arquitectura

3.1 Arquitectura General del Sistema

Para el desarrollo del programa se optó por una arquitectura modular, donde cada funcionalidad fue implementada mediante una función independiente. Esta decisión permitió obtener un código más organizado, fácil de comprender y sencillo de mantener.

El sistema utiliza un menú principal como punto de acceso a todas las funcionalidades. Cada opción seleccionada por el usuario invoca una función específica encargada de realizar una única tarea.

La información se almacena de forma permanente en un archivo denominado **países.csv**, el cual actúa como una pequeña base de datos. Cada vez que el usuario ejecuta una operación, el programa accede al archivo, procesa la información y, cuando es necesario, actualiza su contenido.

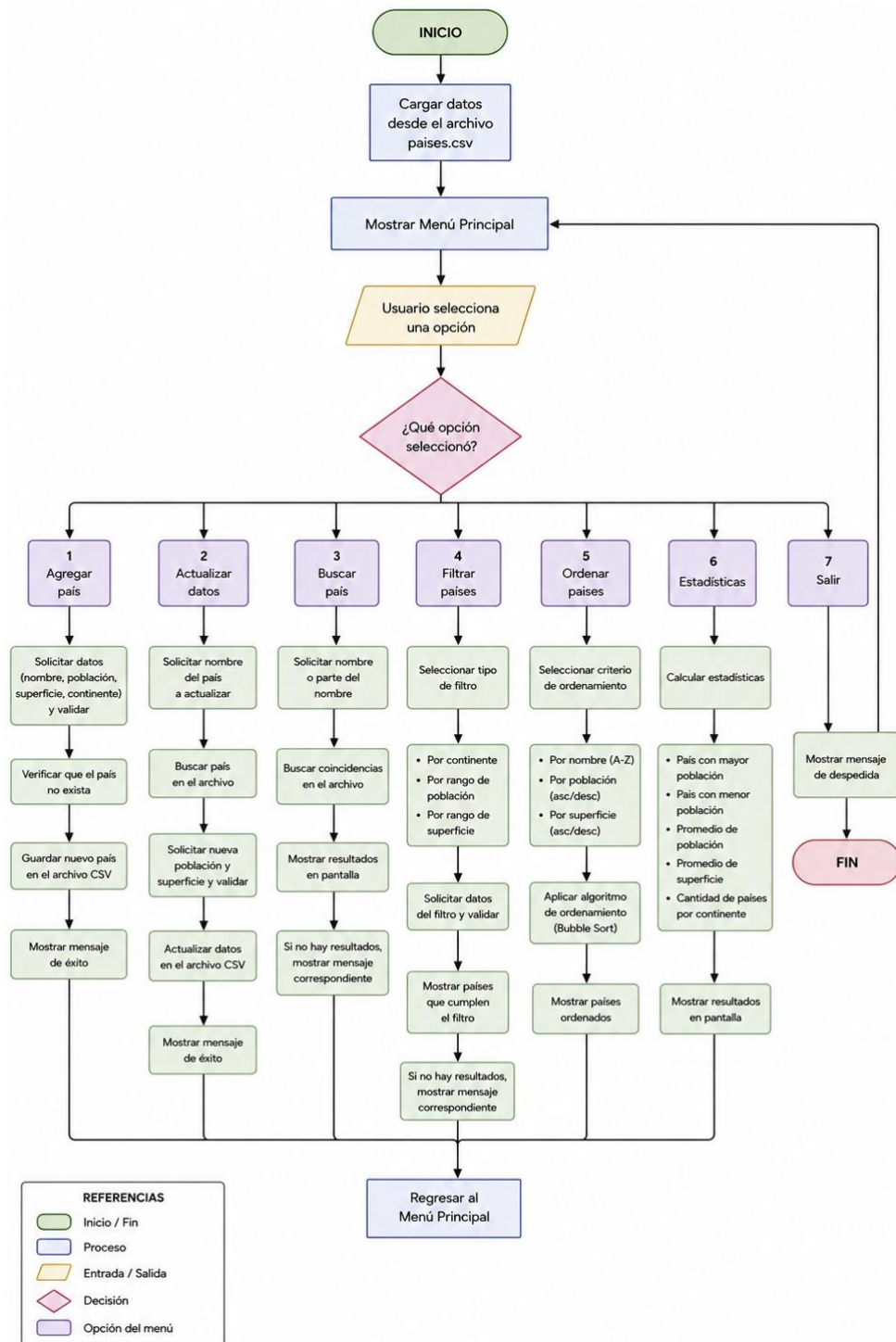
Las principales decisiones técnicas adoptadas durante el desarrollo fueron las siguientes:

- Modularizar el programa mediante funciones independientes.
- Utilizar archivos CSV para almacenar la información de manera persistente.
- Emplear `csv.DictReader()` para facilitar el acceso a los datos mediante claves descriptivas.
- Utilizar `csv.DictWriter()` para reescribir completamente el archivo cuando se actualizan registros.
- Implementar validaciones de entrada para garantizar la integridad de los datos.
- Utilizar listas como estructura temporal para almacenar los registros leídos desde el archivo.
- Implementar el algoritmo Bubble Sort para ordenar los registros sin utilizar funciones avanzadas del lenguaje.

Estas decisiones permitieron desarrollar un sistema simple, organizado y acorde a los contenidos trabajados durante la asignatura.

3.2 Diagrama de Flujo de las Operaciones Principales

El siguiente diagrama representa el funcionamiento general del programa.



El flujo general del sistema comienza con la visualización del menú principal. A partir de la opción seleccionada por el usuario se ejecuta la función correspondiente. Una vez finalizada la operación, el programa retorna nuevamente al menú principal hasta que el usuario selecciona la opción de salir.

3.3 Funcionamiento General del Programa

Cada operación implementada sigue una secuencia similar:

1. Solicitar los datos necesarios al usuario.
2. Validar la información ingresada.
3. Leer el archivo CSV.
4. Procesar los datos según la opción seleccionada.
5. Mostrar el resultado por pantalla.
6. Si corresponde, actualizar el archivo CSV.
7. Regresar al menú principal.

Este flujo permite mantener el programa organizado y garantizar que la información almacenada permanezca consistente durante toda la ejecución.

3.4 Capturas de Pantalla

A continuación se incorporan capturas de pantalla que muestran el funcionamiento del sistema.

Se recomienda incluir al menos las siguientes imágenes:

Figura 1. Menú principal del sistema.

```
===== MENÚ =====  
  
1. Agregar país  
2. Actualizar país  
3. Buscar país  
4. Filtrar  
5. Ordenar  
6. Mostrar estadísticas  
7. Salir  
  
Seleccione una opción: █
```

Figura 2. Agregado de un nuevo país.

```
===== MENÚ =====

1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar
5. Ordenar
6. Mostrar estadísticas
7. Salir

Seleccione una opción: 1
Ingrese el nombre del país: : bolivia
Ingrese la población de bolivia: 10000000
Ingrese la superficie de bolivia: 350000
Ingrese el continente donde se ubica bolivia: : america
País agregado correctamente
```

Figura 3. Actualización de la población y superficie de un país.

```
===== MENÚ =====

1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar
5. Ordenar
6. Mostrar estadísticas
7. Salir

Seleccione una opción: 2
Ingrese el nombre del país: : bolivia
Ingrese la población de bolivia: 15000000
Ingrese la superficie de bolivia: 320000
Se actualizaron los datos ingresados
```

Figura 4. Búsqueda de un país.

```
===== MENÚ =====

1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar
5. Ordenar
6. Mostrar estadísticas
7. Salir

Seleccione una opción: 3
Ingrese el nombre del país: : boli
{'nombre': 'bolivia', 'poblacion': '10000000', 'superficie': '350000', 'continente': 'america'}
```

Figura 5. Filtrado por continente.

```
===== MENÚ =====

1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar
5. Ordenar
6. Mostrar estadísticas
7. Salir

Seleccione una opción: 4

===== MENÚ FILTROS =====

1. Filtrar por continente
2. Filtrar por poblacion
3. Filtrar por superficie

Seleccione una opción: 1
Ingrese el nombre del continente: : america
{'nombre': 'Argentina', 'poblacion': '45376763', 'superficie': '2780400', 'continente': 'America'}
{'nombre': 'Brasil', 'poblacion': '213993437', 'superficie': '8515767', 'continente': 'America'}
{'nombre': 'peru', 'poblacion': '20000000', 'superficie': '1100000', 'continente': 'america'}
{'nombre': 'bolivia', 'poblacion': '15000000', 'superficie': '320000', 'continente': 'america'}
```

Figura 6. Ordenamiento por nombre.

```
===== MENÚ =====

1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar
5. Ordenar
6. Mostrar estadísticas
7. Salir

Seleccione una opción: 5

===== MENÚ ORDENAR =====

1. Ordenar por nombre
2. Ordenar por poblacion
3. ordenar por superficie

Seleccione una opción: 1
{'nombre': 'Alemania', 'poblacion': '83149300', 'superficie': '357022', 'continente': 'Europa'}
{'nombre': 'Argentina', 'poblacion': '45376763', 'superficie': '2780400', 'continente': 'America'}
{'nombre': 'Brasil', 'poblacion': '213993437', 'superficie': '8515767', 'continente': 'America'}
{'nombre': 'Japon', 'poblacion': '190000000', 'superficie': '400000', 'continente': 'Asia'}
{'nombre': 'bolivia', 'poblacion': '15000000', 'superficie': '320000', 'continente': 'america'}
{'nombre': 'peru', 'poblacion': '20000000', 'superficie': '1100000', 'continente': 'america'}
```

Figura 7. Estadísticas del sistema.

```
===== MENÚ =====  
  
1. Agregar país  
2. Actualizar país  
3. Buscar país  
4. Filtrar  
5. Ordenar  
6. Mostrar estadísticas  
7. Salir  
  
Seleccione una opción: 6  
Brasil es el país con mayor poblacion con 213993437 habitantes  
bolivia es el país con menor poblacion con 15000000 habitantes  
La poblacion promedio es 94586583.33333333  
La superficie promedio de los países es 2245531.5  
Países por continente:  
    {'asia': 1, 'america': 4, 'europa': 1, 'africa': 0, 'oceania': 0}
```

Cada una de las capturas permite evidenciar el correcto funcionamiento de las funcionalidades implementadas y verificar el cumplimiento de los requisitos establecidos en la consigna.

4. Dificultades y Conclusiones

4.1 Dificultades Encontradas

Durante el desarrollo del proyecto surgieron diversos desafíos técnicos que requirieron investigación y la aplicación de los conceptos aprendidos durante la cursada.

Una de las principales dificultades fue comprender el funcionamiento del módulo `csv` de Python, especialmente el uso de `DictReader()` y `DictWriter()`. Inicialmente resultó complejo entender cómo leer correctamente los registros del archivo y acceder a cada uno de sus campos mediante claves. Esta dificultad fue superada mediante el estudio de los apuntes de la cátedra y la realización de pruebas hasta comprender el funcionamiento de ambas herramientas.

Otro desafío importante fue la actualización de la información de un país. En un primer momento se intentó modificar directamente el contenido del archivo CSV, pero posteriormente se comprendió que la forma correcta consistía en leer todos los registros, almacenar temporalmente la información en una lista, modificar únicamente el registro correspondiente y reescribir completamente el archivo utilizando `DictWriter()`. Esta solución permitió mantener la integridad de los datos.

También se presentaron dificultades relacionadas con la validación de la información ingresada por el usuario. Fue necesario implementar funciones específicas para controlar que los nombres estuvieran compuestos únicamente por caracteres alfabéticos

y que los valores numéricos fueran enteros positivos. Para ello se utilizaron bloques try-except, evitando que el programa finalizara inesperadamente ante entradas inválidas.

Otra situación que requirió especial atención fue el tratamiento de los datos leídos desde el archivo CSV, ya que todos los valores son interpretados inicialmente como cadenas de texto. Para realizar comparaciones, cálculos estadísticos y ordenamientos fue necesario convertir los campos numéricos al tipo int.

Finalmente, la implementación del algoritmo Bubble Sort para ordenar los países según distintos criterios representó un desafío adicional, ya que implicó comprender el funcionamiento de los recorridos mediante ciclos anidados y el intercambio de posiciones dentro de una lista.

En conjunto, todas estas dificultades fueron resueltas mediante pruebas, depuración del código y la aplicación de los conocimientos adquiridos durante la materia.

4.2 Conclusiones

El desarrollo de este Trabajo Práctico Integrador permitió aplicar de manera práctica los principales contenidos abordados durante la asignatura Programación I. A lo largo del proyecto fue posible integrar conceptos como funciones, estructuras condicionales, estructuras repetitivas, listas, diccionarios, manejo de excepciones y archivos CSV en una aplicación funcional.

Organizar el programa mediante funciones independientes para cada operación facilitó la lectura del código, redujo la repetición de instrucciones y permitió realizar modificaciones sin afectar el resto del sistema.

Asimismo, el proyecto permitió comprender la importancia de validar la información ingresada por el usuario y de mantener una correcta organización de los datos almacenados.

En conclusión, este trabajo integrador permitió consolidar los conocimientos adquiridos durante la cursada y evidenció la importancia de una correcta planificación, modularización y validación en el desarrollo de aplicaciones de software.

5. Bibliografía / Webgrafía

Durante el desarrollo del presente Trabajo Práctico Integrador se consultaron las siguientes fuentes de información:

- Material de estudio y clases de la asignatura **Programación I**, Tecnicatura Universitaria en Programación a Distancia (TUPAD), Universidad Tecnológica Nacional – Facultad Regional San Nicolás.
- Material de estudio de la asignatura **Organización Empresarial**, Tecnicatura Universitaria en Programación a Distancia (TUPAD), Universidad Tecnológica Nacional – Facultad Regional San Nicolás.
- Anthropic. (2026). *Claude*. Disponible en: <https://claude.ai/>
- OpenAI. (2026). *ChatGPT*. Disponible en: <https://chatgpt.com/>